



Основы работы с Docker

Microservice Architecture



Проверить, идет ли запись

**Меня хорошо видно
& слышно?**





Вадим Перепелкин

- Более 15 лет в ИТ
- Занимал разные позиции (developer, team lead, architect)

 @VPerepelkin

 vup@rambler.ru

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#канал группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



Основы Docker

Dockerfile

Под капотом

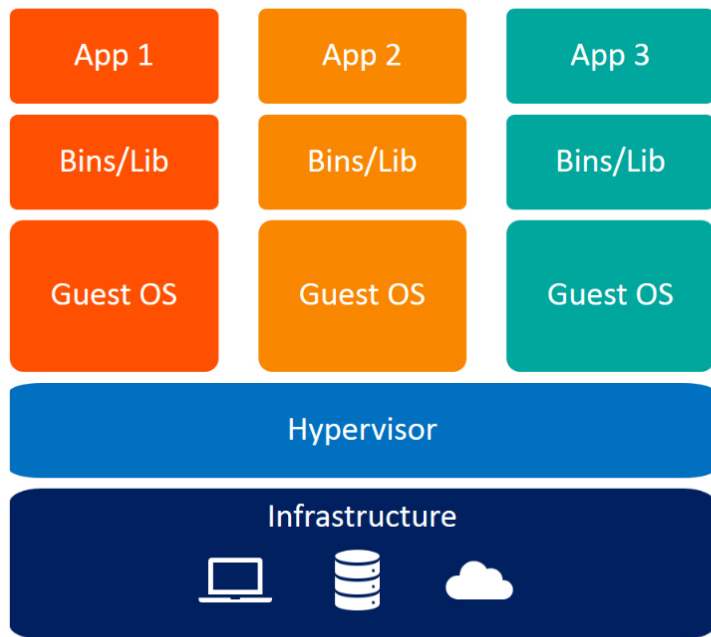
Цели вебинара

К концу занятия вы сможете

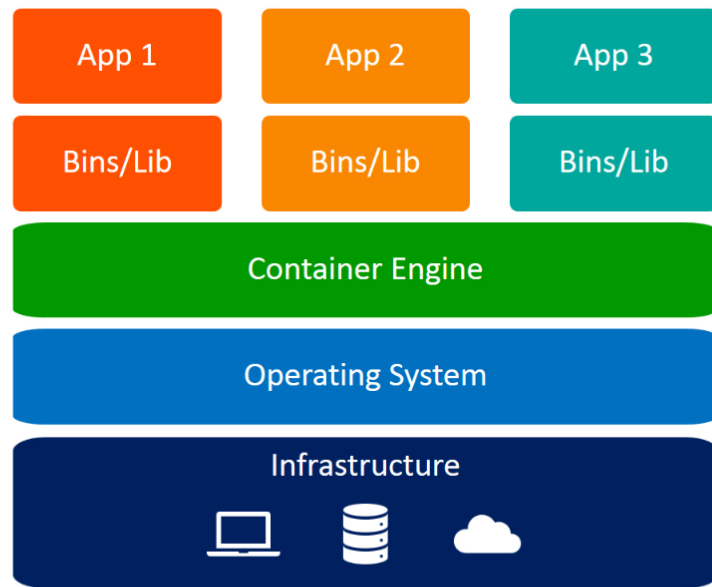
1. собрать свой докер-образ
2. запускать докер-контейнер из образа
3. поймете, как это работает под капотом

Docker

Контейнеризация и виртуализация



Virtual Machines

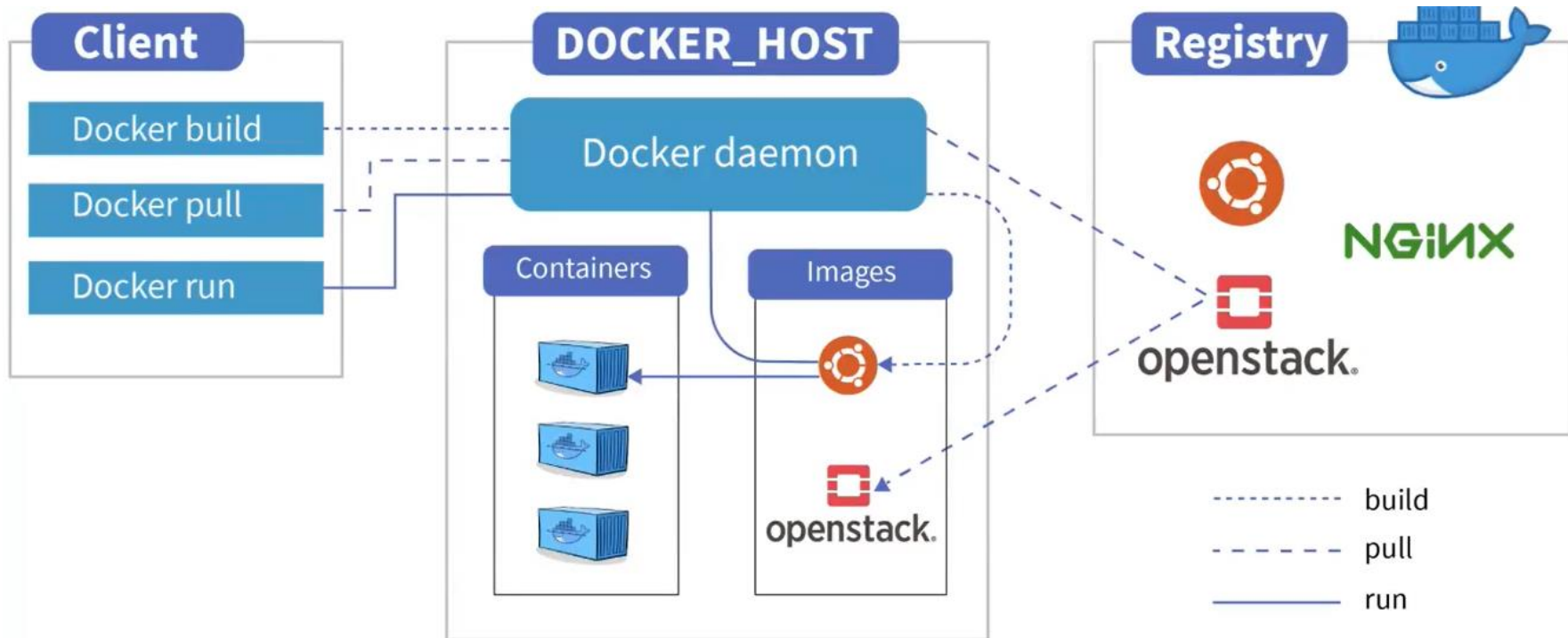


Containers

Почему Docker?

- Абстракция от host-системы
- Легковесные изолированные окружения
- Общие слои файловой системы
- Компоновка и предсказуемость
- Простое управление зависимостями
- Дистрибуция и тиражируемость
- Стандартизация описания окружения, сборки
- 100% консистентная среда приложения
- Воспроизводимость

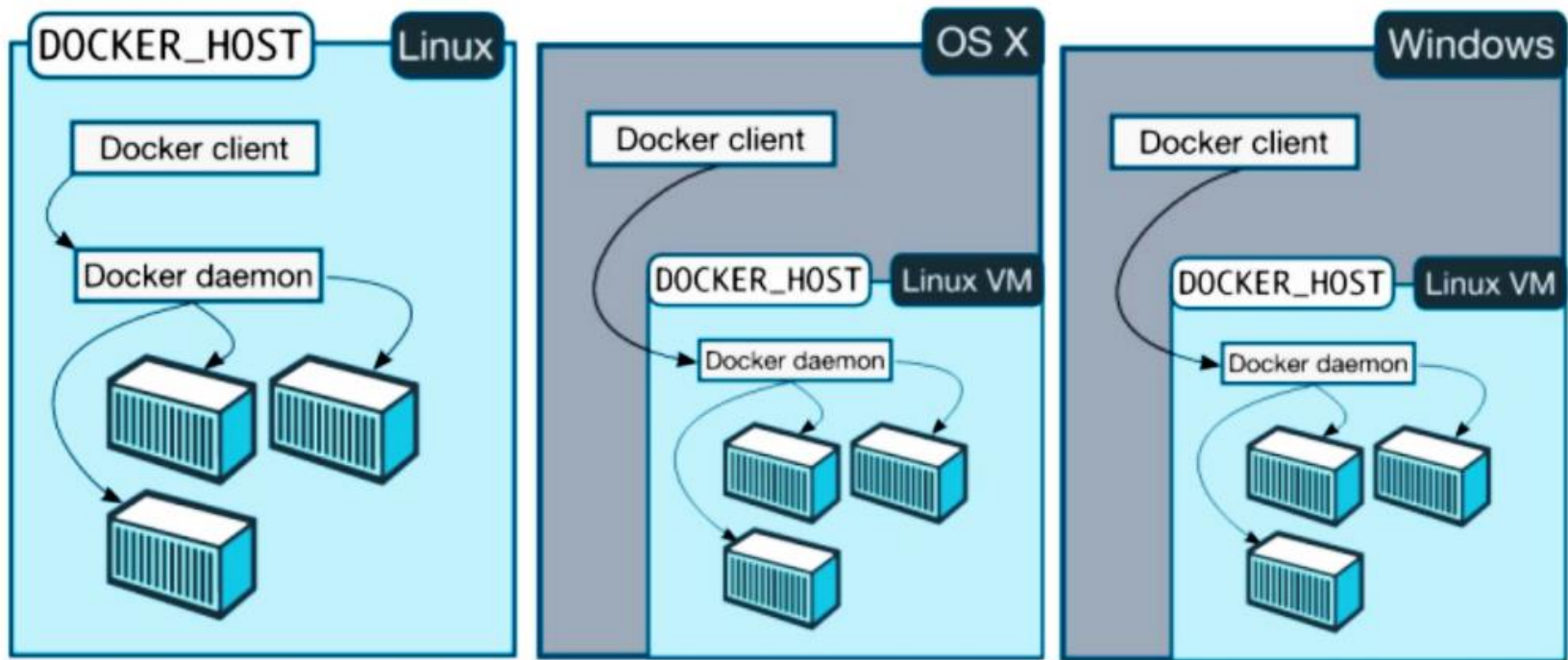
Как это работает



Контейнеризация

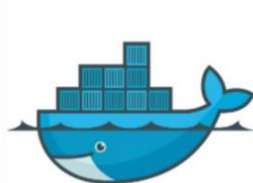
- Контейнеры эфемерны, но можно монтировать volumes в контейнер
- Контейнер завершает свою работу, когда завершается основной процесс внутри контейнера
- Один контейнер - одно приложение
- Логи пишутся в STDOUT и STDERR
- Используется ядро хостовой ОС

Контейнеризация



Docker-registry –

это репозиторий, в котором хранятся Docker-образы. Он может быть как локальным, так и публичным. Репозитории создают на платформах вроде Docker Hub и GitLab и размещают в них образы с описанием, разными версиями и тегами



GitLab



Вопросы?

 если есть вопросы

 если вопросов нет

Dockerfile и сборка образа

Dockerfile

- Текстовый файл с build инструкциями
- Инструкции декларативно описывают image
- Каждая инструкция - промежуточный image
- Сам build делает docker daemon

Команды

- FROM – задает базовый (родительский) образ
- ENV – переменные окружения
- ARG – переменные во время сборки
- COPY – скопировать файл или папку
- RUN – выполнить команду
- ADD – скопировать файл или папку, скачать по ссылке, разархивировать архив
- EXPOSE – документация
- CMD – определяет команду запуска
- ENTRYPOINT – определяет команду запуска

Пример Dockerfile

 Dockerfile2

```
1 FROM alpine:3.12
2 RUN apk update && apk upgrade
3 RUN wget https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
4 RUN unzip terraform_0.12.28_linux_amd64.zip
5 RUN rm terraform_0.12.28_linux_amd64.zip
6 RUN mv terraform /usr/local/bin/
7 RUN wget https://github.com/terraform-linters/tflint/releases/download/v0.16.2/tflint_linux_amd64.zip
8 RUN unzip tflint_linux_amd64.zip
9 RUN rm tflint_linux_amd64.zip
10 RUN mv tflint /usr/local/bin/
11 RUN apk add curl
12 CMD ["/bin/sh"]
```

CMD vs ENTRYPOINT

- Можно использовать по отдельности и аналогично
- Можно использовать совместно - тогда CMD определяет дополнительные параметры запуска

```
FROM alpine
ENTRYPOINT ["ping", "www.google.com"]
```

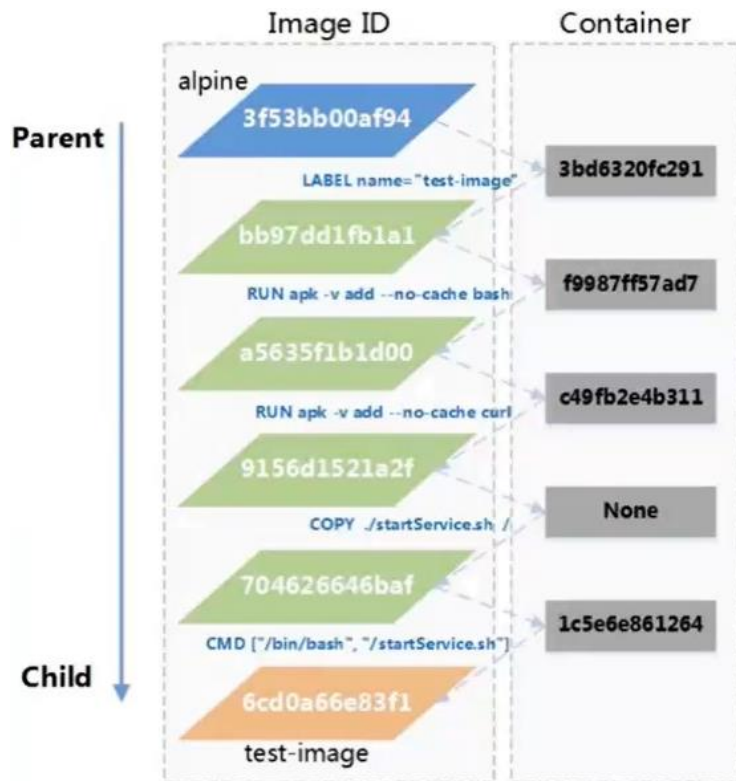
```
FROM alpine
ENTRYPOINT ["ls", "/usr"]
CMD ["/var"]
```

Shell vs EXEC

```
FROM alpine  
ENTRYPOINT ping www.google.com
```

```
FROM alpine  
ENTRYPOINT ["ping", "www.google.com"]
```

Docker-образ



Слои

```
# Базовый образ ~6mb
FROM alpine:latest
# Open JRE ~180mb
RUN apk update && apk add openjdk21-jre
WORKDIR /app
# Jar ~10mb
COPY app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Базовый образ ~6mb
FROM alpine:latest
WORKDIR /app
# Jar ~10mb
COPY app.jar app.jar
# Open JRE ~180mb
RUN apk update && apk add openjdk21-jre
ENTRYPOINT ["java", "-jar", "app.jar"]
```



LIVE



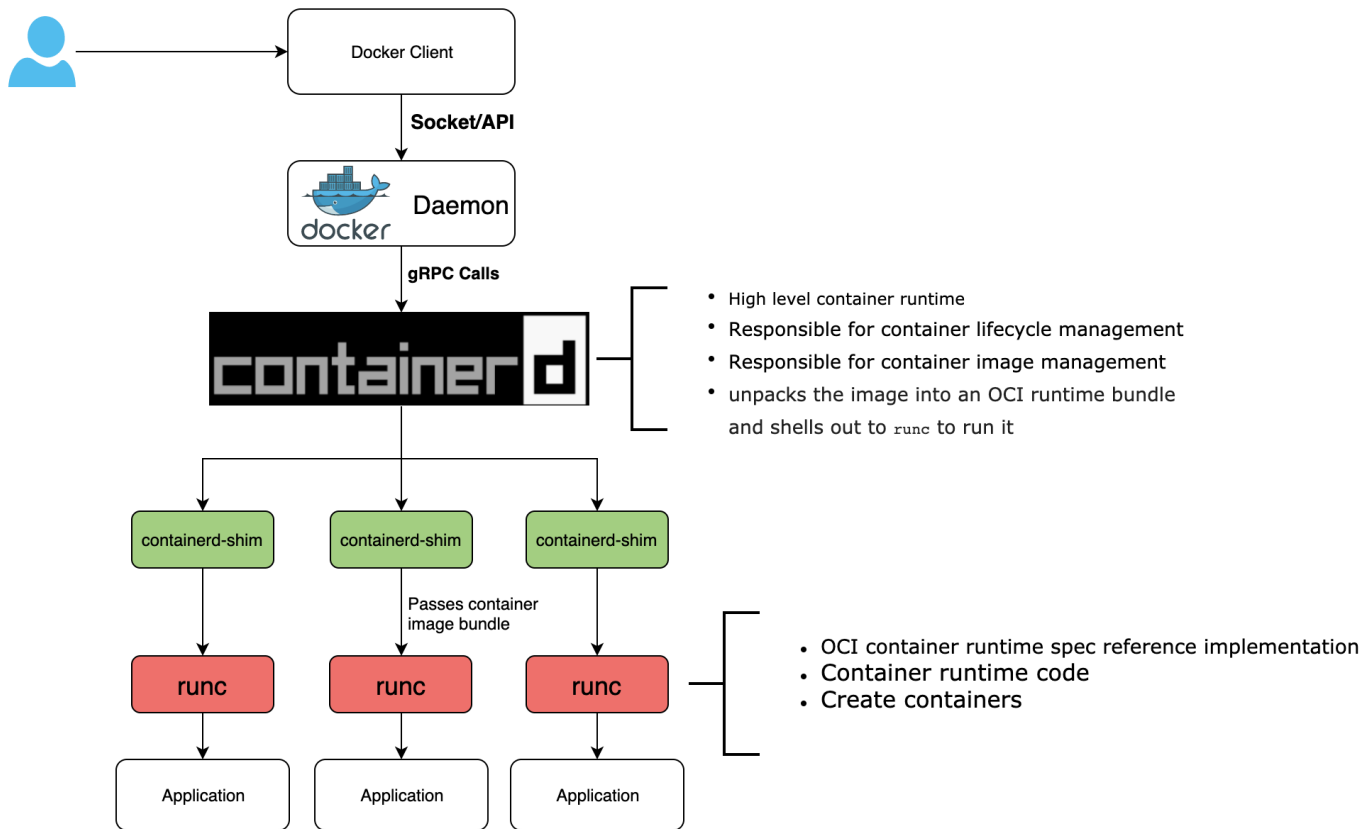
Вопросы?

 если есть вопросы

 если вопросов нет

Docker под капотом

Как это работает



Как реализуется контейнеризация

- Namespaces
- Cgroups
- UnionFS

Namespaces

- PID (Process ID)
- Mnt (mount)
- Net (network)
- Uts (Unix Time Sharing)
- IPC (Inter-process communication)
- user

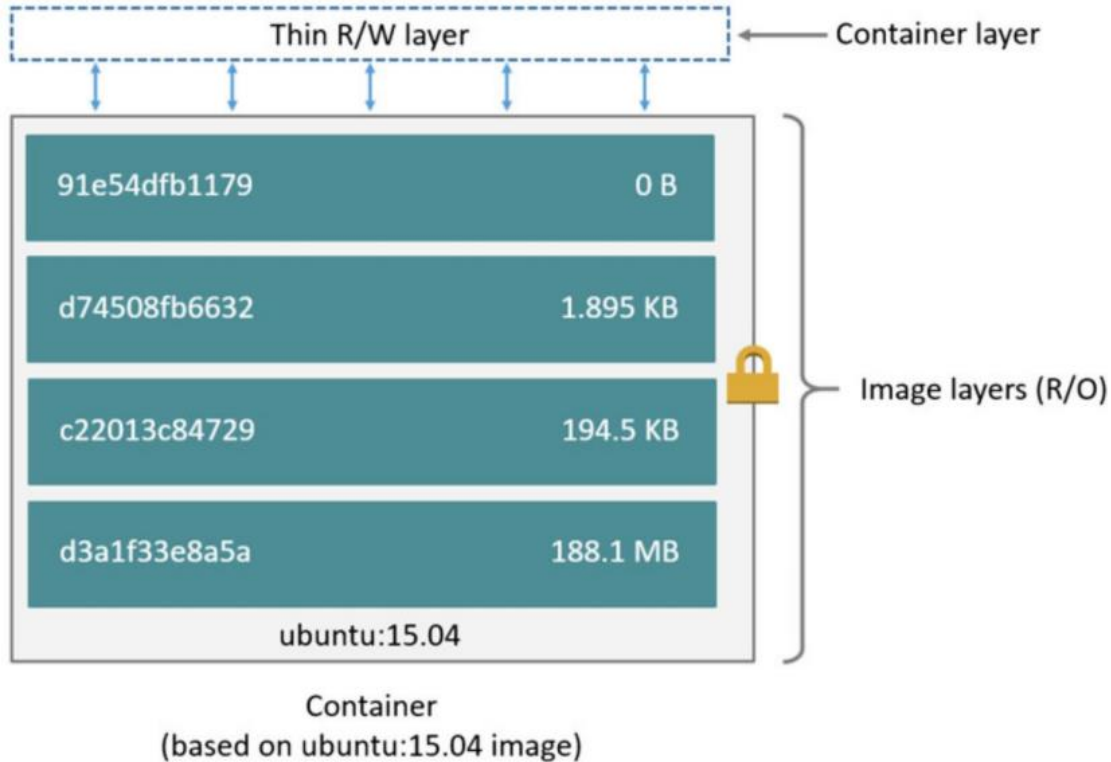
Cgroups

- Ограничивает доступ к ресурсам (в т.ч. устройствам)
- Ограничивает доступ к системным вызовам

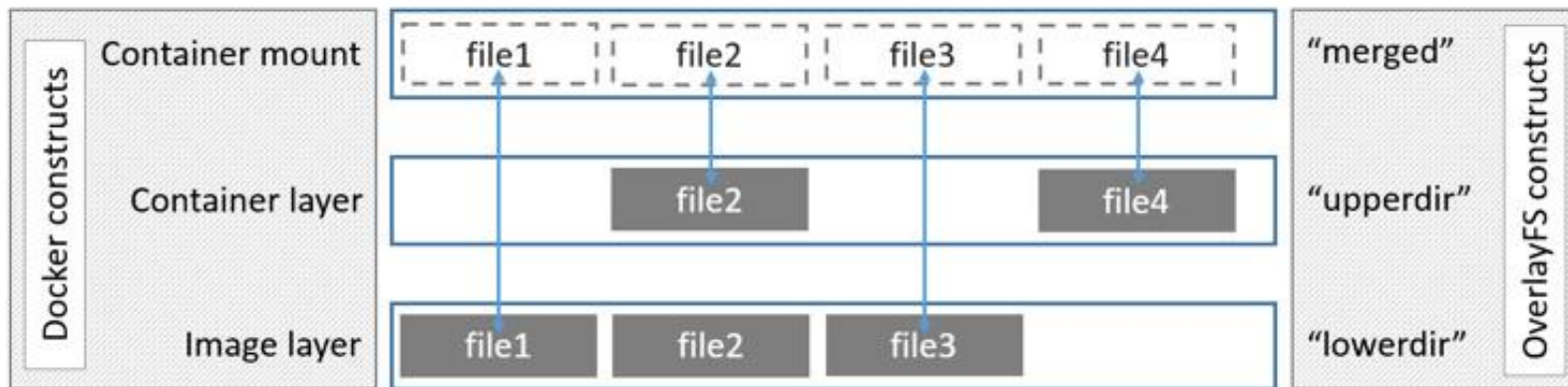
UnionFS

- Обеспечивает хранение слоев образов
- Обеспечивает слой для контейнера
- Драйвер выбирается в зависимости от потребностей (но обычно нет необходимости)

UnionFS

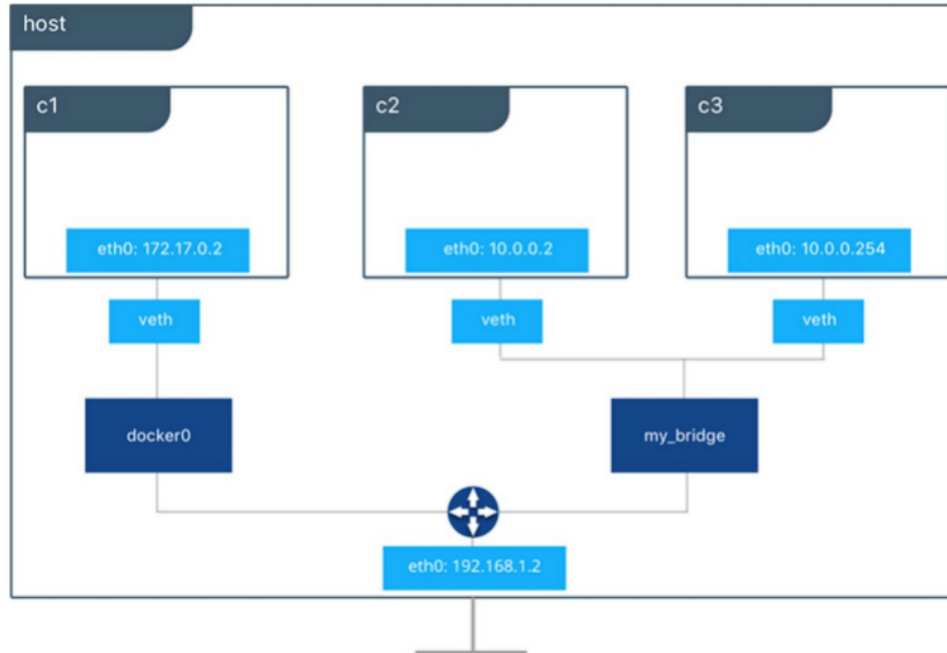


UnionFS



Docker network. Bridge

Bridge



Docker network. Какие еще есть сетевые драйверы

- Bridge (по умолчанию)
- Host
- None
- Macvlan
- Overlay
- IPvlan

Docker volumes

- mounts
- Docker volumes
- tmpfs



LIVE



Вопросы?

 если есть вопросы

 если вопросов нет

**Подведём
итоги занятия**

Цели вебинара

Отправьте в чат номера достигнутых целей

1. собрать свой докер-образ
2. запускать докер-контейнер из образа
3. понимать как это работает под капотом

Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Приглашаем на следующий вебинар: “Архитектура Kubernetes”



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой